



ESCUELA DE
INGENIERÍA EN CIENCIAS Y SISTEMAS
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



Dia, Fecha:	Sabado, xx/xx/2025
Hora de inicio:	XX:XX

Laboratorio de Programacion de computadoras I, Sección X

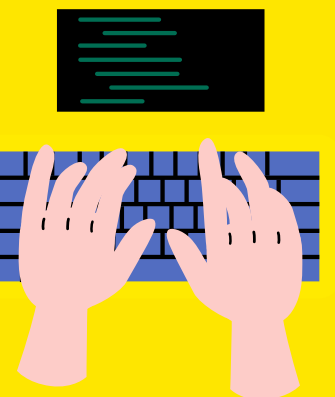
Nombre del estudiante

Clase 11

PRÁCTICA DE PROGRAMACIÓN DE COMPUTADORAS

SECCIÓN X

CÓDIGO DEL CURSO 090



AGENDA DE LA CLASE:



Review Conferencia

Teoria VBA

Tarea 4

Foro de la semana

¿Dudas?





LIVE

¿Qué es VBA Excel?

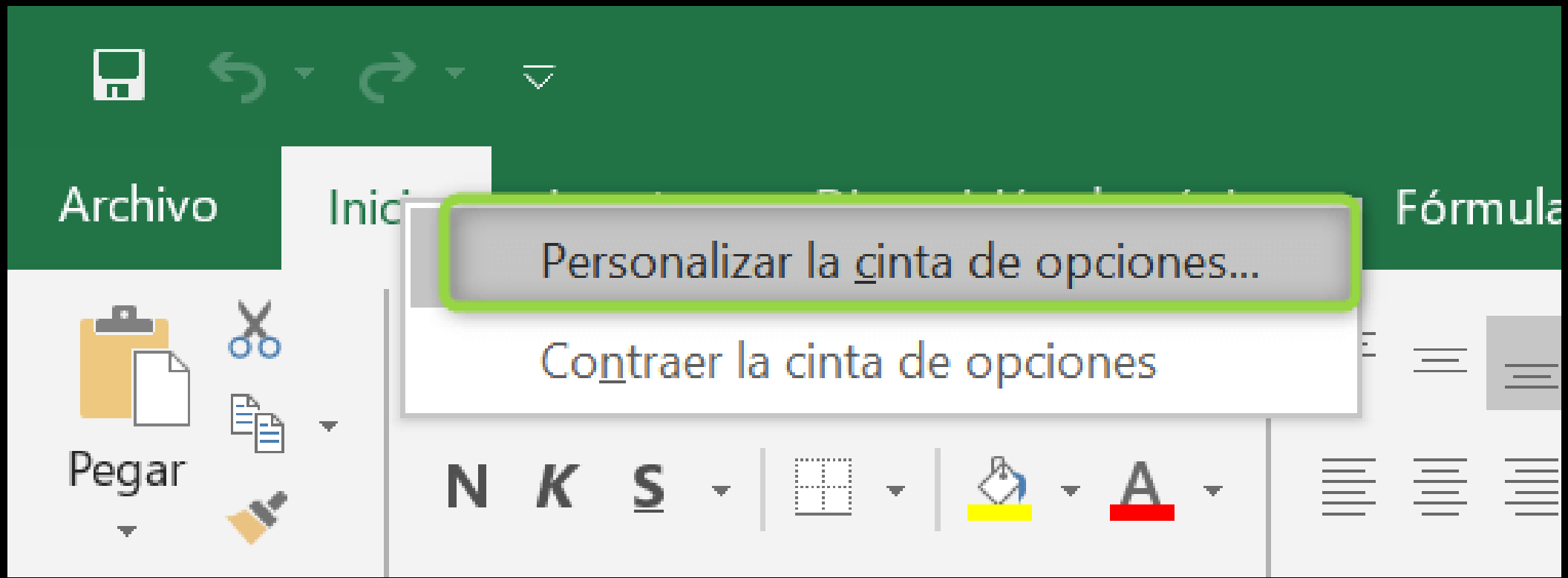


VBA es el lenguaje de programación utilizado por Microsoft para crear aplicaciones en Windows. Viene integrado en la suite de Office: Outlook, Word, Power Point...etc. A continuación vamos a ver cómo empezar a utilizarlo para crear pequeños programas en Excel.

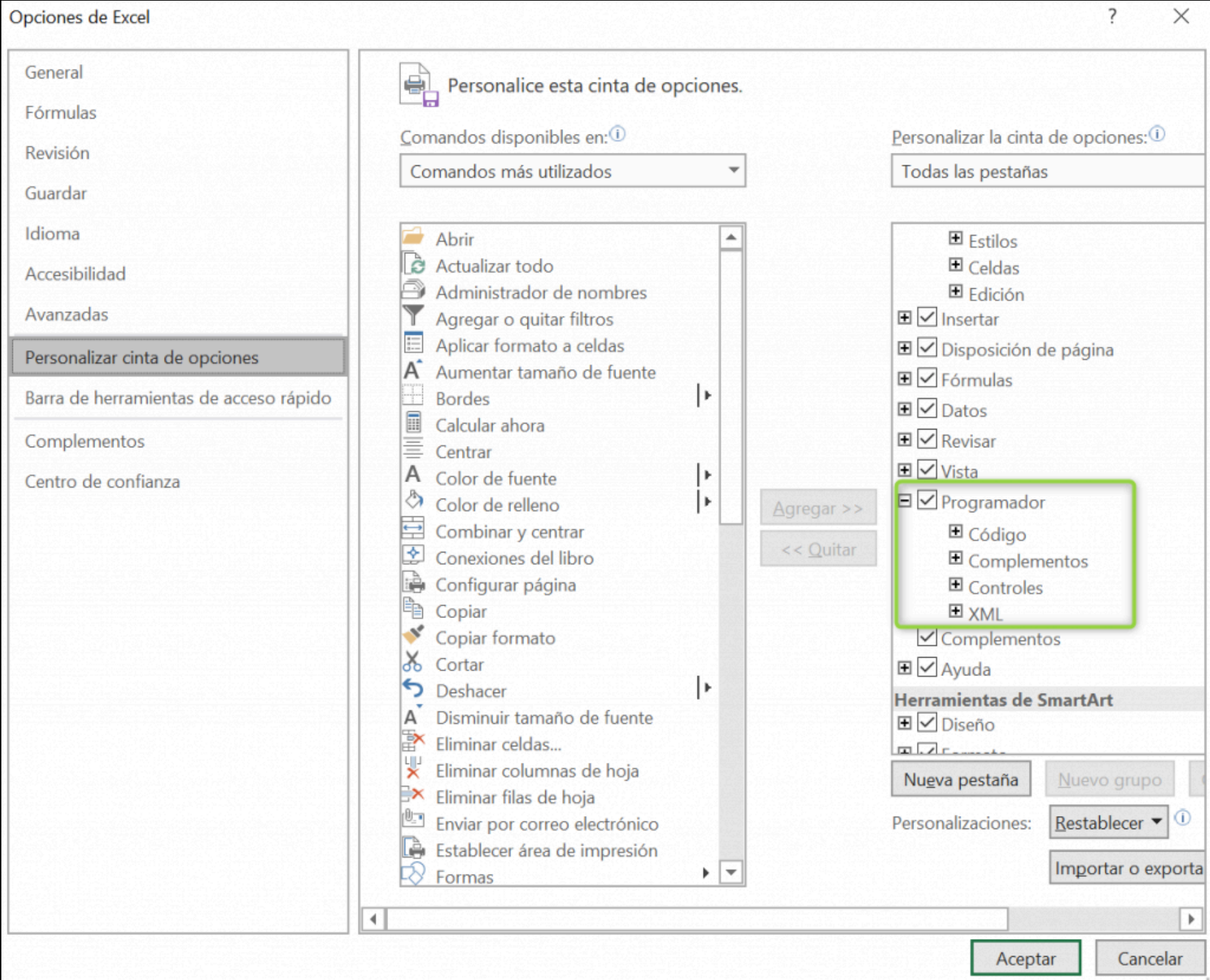


Editor de Visual Basic Application en Excel

Para poder empezar a programar en VBA necesitamos habilitar la pestaña Programador. Para hacerlo, haz clic derecho en cualquier elemento del menú superior y haz clic en Personalizar la cinta de opciones.



Llegarás a una ventana donde debes hacer clic en Personalizar cinta de opciones (en el menú de la izquierda) y seleccionar a la derecha la opción Programador.



Hecho esto ya te aparecerá el menú con las opciones necesarias para crear tus primeros programas y macros.



Variables

Las variables son la representación de un valor o un objeto. Se declaran utilizando Dim, ejemplo:

```
1 Dim nombreVariable As tipodatos
2
```

Tipos de datos en VBA

Tipo	Datos que contiene
Boolean	True o False
Byte	Nº enteros entre 0 y 255
Integer	Nº enteros entre -32768 y 32767
Long	Nº enteros entre -2147483648 y 2147483647
Single	Nº decimales de precisión simple
Double	Nº decimales de precisión doble
String	Cadenas de caracteres
Date	Fechas
Object	Referencias a objetos
Variant	Cualquier tipo de dato



Expresiones u operadores

Las expresiones en VBA aritméticas, relacionales o lógicas. A su vez, existen operadores que permiten componer este tipo de expresiones. Los operadores son:

OPERADOR	OPERACIÓN QUE EFECTÚA
Aritméticos	
+	Suma
-	Resta
\	División entera
Mod	Resto de la división entera
*	Multiplicación
/	División
^	Potencia del primer operando



Expresiones u operadores

OPERADOR	OPERACIÓN QUE EFECTÚA
Relacionales	
=	Comprueba si los dos operandos son iguales
<>	Comprueba si los dos operandos son distintos
<=	Comprueba si el primero es menor o igual que el segundo
>	Comprueba si el primero es mayor que el segundo
>=	Comprueba si el primero es mayor o igual que el segundo



Expresiones u operadores

OPERADOR	OPERACIÓN QUE EFECTÚA
Lógicos	
And	Devuelve true si los dos son true
Or	Devuelve true si uno de los dos, o los dos son true
Xor	Devuelve true si uno de los dos es true y el otro false
Eqv	Devuelve true si los dos son true o false
Not	Devuelve true si el operando es false y viceversa



Sub rutinas

Los procedimientos Sub siempre comienzan con la palabra reservada Sub, el nombre de la macro (cada macro debe tener un nombre único) y un par de paréntesis (estarán vacíos a menos que el procedimiento utilice uno o más argumentos).

La instrucción End Sub señala el final del procedimiento. Las líneas que se encuentran en medio son el código del procedimiento.

Un ejemplo de sub rutina:

```
Sub FechaActual()  
    'Inserta la fecha actual en la celda activa al hacer clic en un  
    ActiveCell.Value = Date  
    ActiveCell.NumberFormat = "mmmm d, yyyy"  
    ActiveCell.Font.Bold = True  
    ActiveCell.Columns.AutoFit  
End Sub
```



Funciones

Una función siempre devuelve un único valor (del mismo modo que una función de hoja siempre devuelve un único valor). Una función VBA puede ejecutarse desde otro procedimiento VBA o usarse en fórmulas de hoja, igual que usaría las funciones de hoja integradas en Excel.

Ejemplo: Script para extraer el dominio de una url:

```
1  Function rcStrDomain(url As String) As String
2
3      'Comprobamos si el parámetro url es verdaderamente una url
4      If Left(url, 4) Like "http*" Then
5          'Extraemos todo lo que va después de //
6          url = Mid(url, InStr(url, "//") + 2)
7      Else
8          'Si no es una url, lo introducimos en la celda
9          rcStrDomain = "No es una url"
10         Exit Function
11     End If
12     'Comprobamos si el subdominio contiene las tres típicas www
13     If Left(url, 4) Like "[Ww][Ww][Ww0-9]." Then
14         'De ser así, extraemos lo que va después de www (o varia
15         url = Mid(url, 5)
16     End If
17     'Finalmente, la función devuelve la cadena de texto hasta qu
18     rcStrDomain = Split(url, "/")(0)
19
20 End Function
```



Condicionales

Las sentencias condicionales en Visual Basic funcionan de forma muy parecida a otros lenguajes de programación. Encontramos dos tipos de sentencias de ramificación, una de las cuales tiene tres variaciones:

La sentencia If simple

```
1 | If condición Then
2 |     sentencias
3 | End If
```

Añadir la sentencias Else

```
1 | If condición Then
2 |     sentencias1
3 | Else
4 |     sentencias2
5 | End If
```

La sentencia ElseIf

```
1 | If condición1 Then
2 |     sentencias1
3 | ElseIf condición2 Then
4 |     sentencias2
5 | Else
6 |     sentencias3
7 | End If
```





Condicionales

```
Select Case expresión
  Case expresión1
    sentencias1
  Case expresión2
    Sentencias2
  .....
  Case expresiónN
    sentenciasN
  Case Else
    sentencias para la alternativa no coincidente con ninguna
End Select
```



Bucles

Los bucles son la solución para cuando necesitamos repetir una serie de instrucciones un número de veces para resolver un problema.

Existen tres tipos de bucles, uno de los cuales tiene cuatro variedades:

```
1  'Mientras se cumpla una condición
2  While condición
3      'Ejecuta una acción
4      sentencias
5  Wend
6
7  'Ejemplo
8  Dim Contador
9  Contador = 0 ' Valor inicial.
10 While Contador < 20 ' Mientras sea menor de 20.
11     Contador = Contador + 1 ' Incrementa su valor +1.
12 Wend ' El bucle termina cuando el Contador > 19.
13 MsgBox Contador ' Mensaje con el valor de Contador, igual a 20.
```



Bucles

Bucle Do ... Loop

```
1  'Ejecuta una acción
2  Do
3  Sentencias
4  'Mientras se cumpla una condición
5  Loop While/Until condición
6
7  'Ejemplo
8  Dim index As Integer = 0
9  Do While index <= 10 'Mientras el valor sea menor igual que 10
10     Debug.Write(index.ToString & " ") 'Convierte el valor a strin
11     index += 1 ' Aumenta el valor del contador +1
12 Loop
13
14 Debug.WriteLine("")
15 ' Resultado: 0 1 2 3 4 5 6 7 8 9 10
```



Bucles

Bucle For ... Next

```
1  'Repite una acción un nº limitado de veces. Desde un valor hasta  
2  For variable = expresión1 To expresión2 [Step expresión3]  
3      'Ejecuta una acción  
4      Sentencias  
5  Next variable
```



Objetos

Excel cuenta con decenas de objetos que representan a todos los elementos que podemos imaginar: estilos, nombres definidos para celdillas y rangos, gráficos, libros, hojas, rangos, ventanas, complementos y proyectos de Visual Basic son algunos de ellos.

El objeto que actúa como raíz de todos los demás, del cual dependen, es **Application**.

El libro

Cada libro de Excel, que hasta hemos utilizado en calidad de usuarios, es la representación de un objeto Workbook. Partiendo del objeto Application podemos acceder tanto al libro actual, con la propiedad ActiveWorkbook, como a la colección de todos los libros abiertos, con la colección Workbooks.



La hoja

Cada libro contiene una o más hojas de cálculo, cada una de las cuales tiene correspondencia directa con un objeto Worksheet. A través de este objeto, con sus propiedades y métodos, podemos acceder a todo el contenido que exista en la hoja: celdillas, gráficos, elementos visuales como los diagramas, tablas dinámicas, etc.

Los rangos

Un rango puede ser una celda, una fila, una columna o una agrupación de cualquiera de éstos. El objeto Range es probablemente el objeto más frecuentemente utilizado en Excel VBA



¿Cómo hacer referencia a rangos en otras hojas?

```
1 | Worksheets("Hoja1").Range("A1")
```

¿Cómo utilizar la propiedad Cells para seleccionar un rango?

```
1 | Cells(5, "C")
```

¿Cómo utilizar la propiedad Offset para hacer referencia a un rango?

```
1 | Range("A1").Offset(4, 5)
```



Desplazamientos

INDICADOR	DESPLAZAMIENTOS
xlDown	Hacia abajo
xlUp	Hacia arriba
xlToRight	Hacia la derecha
xlToLeft	Hacia la izquierda



Clase 11



**¿Tienes alguna
pregunta?**



Espero, que hayas aprendido algo nuevo.

